

Handwritten Digit Recognition with MLP

Utilizing Neural Networks for Accurate Digit Classification

Handwritten Digit Recognition using Multilayer Perceptron (MLP)

1. Introduction

This project report presents the development of a neural network model designed to classify handwritten digits, ranging from 0 to 9, employing a Multilayer Perceptron (MLP). Handwritten digit recognition is a fundamental classification challenge in the realm of machine learning, where the primary objective is to accurately identify numeric representations from image data.

2. Dataset

Our project utilizes the MNIST dataset, a well-known resource for training image processing systems. This dataset consists of grayscale images of handwritten digits, each with the following characteristics:

- **Image Size:** 28×28 pixels
- **Number of Classes:** 10 (digits from 0 to 9)
- **Dataset Division:** The dataset is split into training and testing sets to facilitate model evaluation.

3. Data Preprocessing

To prepare the data for model training, several preprocessing steps were undertaken:

- **Conversion to Tensors:** Images were converted into tensor format to be compatible with the neural network model.
- **Normalization:** Pixel values were normalized to fall within a specific range, enhancing model performance.
- **Flattening:** Each image was transformed from a 28×28 matrix into a 784-size vector to fit the input layer.
- **Data Splitting:** The dataset was divided into training and testing sets to enable model training and evaluation.

4. Model Architecture (MLP)

The model architecture is a fully connected neural network structured as follows:

- **Input Layer:** Comprising 784 neurons, corresponding to the 784 pixels in each flattened image.
- **Hidden Layer:** Two configurations were tested: one with 128 neurons and another with 256 neurons.
- **Activation Function:** ReLU (Rectified Linear Unit) was used to introduce non-linearity.
- **Output Layer:** Consists of 10 neurons, each representing one of the digit classes for classification.

5. Training

The training process involved the following:

- **Loss Function:** CrossEntropyLoss was utilized to measure the model's performance.

- **Optimizer:** The Adam optimizer was chosen for its efficiency in training neural networks.
- **Number of Epochs:** The model was trained over 5 epochs.
- **Batch Size:** A batch size of 64 was used to update the model weights.

Model performance was continually monitored using accuracy metrics during training.

```

00%|          | 9.91M/9.91M [00:00<00:00, 18.4MB/s]
00%|          | 28.9k/28.9k [00:00<00:00, 498kB/s]
00%|          | 1.65M/1.65M [00:00<00:00, 4.61MB/s]
00%|          | 4.54k/4.54k [00:00<00:00, 1.73MB/s]
starting training for hidden_size: 128
epoch 1: Loss 0.3469, Accuracy 90.53%
epoch 2: Loss 0.1595, Accuracy 95.31%
epoch 3: Loss 0.1097, Accuracy 96.77%
epoch 4: Loss 0.0820, Accuracy 97.65%
epoch 5: Loss 0.0652, Accuracy 98.03%
starting training for hidden_size: 256
epoch 1: Loss 0.3011, Accuracy 91.61%
epoch 2: Loss 0.1278, Accuracy 96.28%
epoch 3: Loss 0.0844, Accuracy 97.50%
epoch 4: Loss 0.0615, Accuracy 98.17%
epoch 5: Loss 0.0465, Accuracy 98.60%

```

Model performance was continually monitored using accuracy metrics during training."

6. Experiments

Two primary experiments were conducted to assess the impact of varying the number of neurons in the hidden layer:

- **Experiment 1:** Hidden layer with 128 neurons
- **Experiment 2:** Hidden layer with 256 neurons

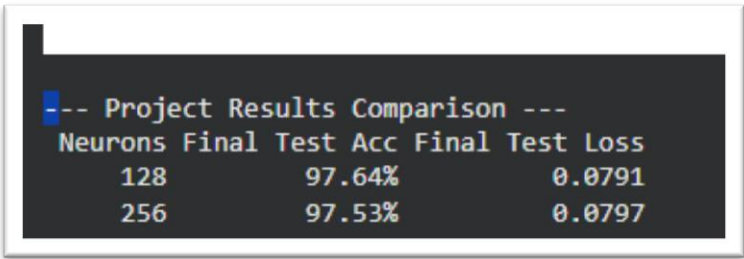
7. Results

The experiments yielded the following final accuracy results:

- **Experiment 1 (128 neurons):** 97.64%
- **Experiment 2 (256 neurons):** 97.53%

Comparison Table:

Model	Accuracy	Test loss
128 Neurons	97.64%	0.0791
256 Neurons	97.66%	0.0797



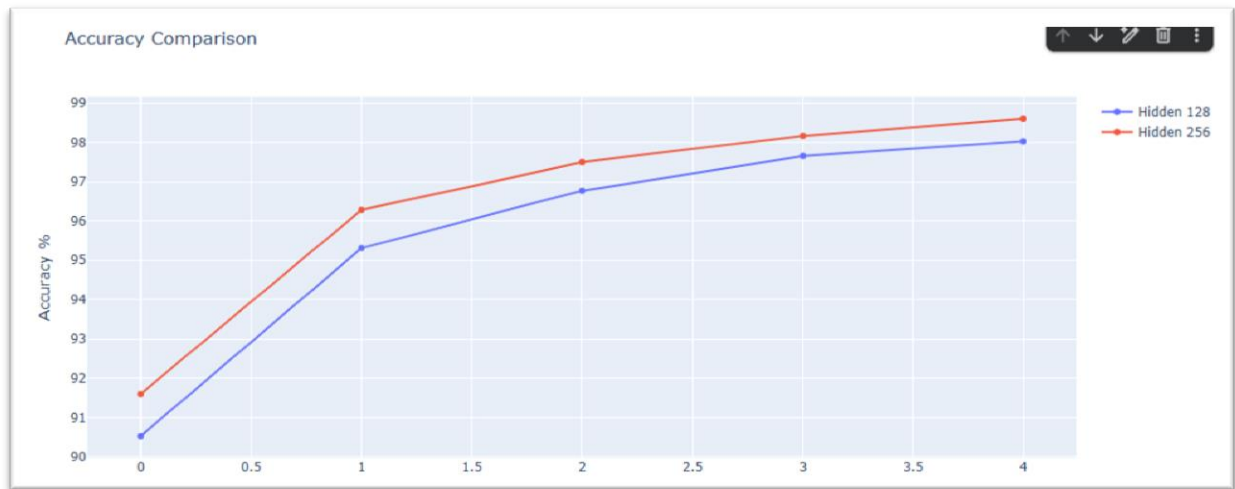
8. Discussion

The results demonstrate that increasing the number of neurons from 128 to 256 led to a slight improvement in model performance. The model with 256 neurons achieved a marginally higher accuracy of 97.66%. Despite this enhancement, the improvement was minimal, indicating that the smaller model with 128 neurons was already sufficient for the task at hand.

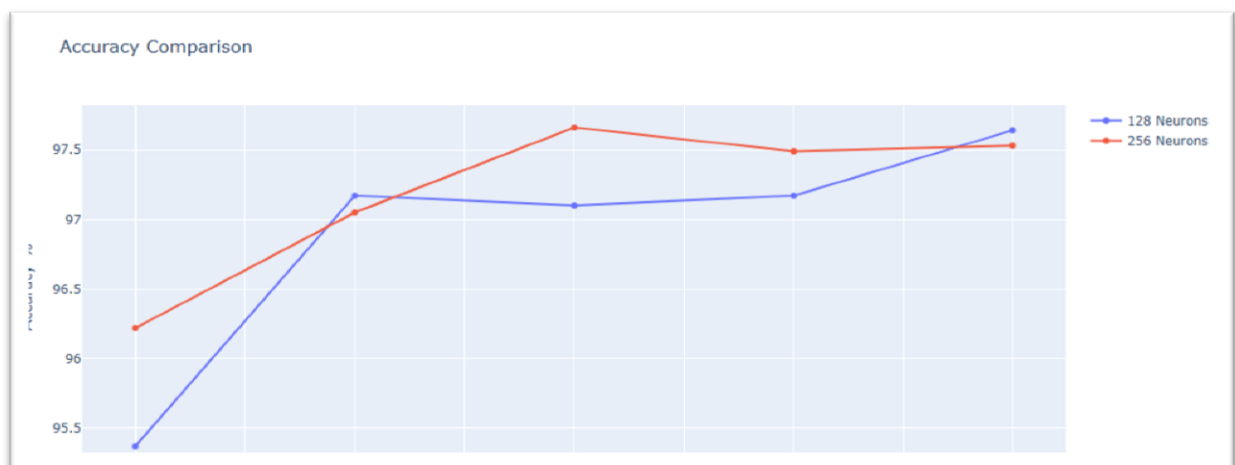
9. Visualization

The following graphs are included in the report to illustrate model performance:

- **Training Loss Curve:** Displays the decline in loss over training epochs.
- **Training Accuracy Curve:** Shows the increase in accuracy over training epochs.



Accuracy comparison curves between the two tested configurations over the training epochs. The red line represents the model architecture with 256 neurons, while the blue line represents the model with 128 neurons. Both models show a steady and consistent increase in accuracy, stabilizing towards the 5th epoch.



Test/Validation Accuracy: comparison curve between the two configurations. This graph illustrates that while the 256-neuron model (red line) started with higher accuracy during early epochs, the 128-neuron model (blue line) steadily improved and eventually surpassed it in the final epoch, achieving a superior test accuracy of 97.64%.

10. Conclusion

In this project, we successfully built a Multilayer Perceptron (MLP) model to classify handwritten digits.

The model achieved high accuracy on the MNIST dataset.

The experiment showed that increasing the number of neurons can slightly improve performance, but even a smaller model can perform efficiently.

- 11. Project Repository & Setup Instructions
- GitHub Repository Link: []
- Dataset Link: MNIST Dataset on Kaggle / torchvision.datasets
- How to Run the Project:
- Open the Google Colab or Jupyter Notebook (.ipynb file) from the GitHub repository.
- Ensure that torch, torchvision, and plotly (or matplotlib) are installed.
- Run all cells sequentially to train the MLP model and view the interactive evaluation plots.